File: `src/components/MinMaxReadout.jsx`

Category: React Component (Statistics Readout: Min/Max)
Runtime: Browser (React)
Primary parent: `src/components/ShellyDashboard.jsx`
Primary inputs: historical data array + selected time interval
Upstream dependencies: `src/config/seriesCatalog.js` (metric definitions + grouping), `src/styles/uiStyles.js` (card styling)

Overview

`MinMaxReadout` computes and displays minimum and maximum values for each metric:

- **Overall min/max** (using the full `dataHistory`)
- **Interval min/max** (using only the last `selectedInterval` milliseconds)

It is a read-only statistics component. It does not fetch data and does not change application state.

Imports

- React: `useMemo`
- `SERIES_CATALOG`, `ALL_METRIC_KEYS`, `getGroupedMetricKeys` from `../config/seriesCatalog`
- `cardStyle` from `../styles/uiStyles`

Internal Helper Functions

`calculateMinMax(data, valueAccessor)`

Signature

```
calculateMinMax(
  data: ParsedDataPoint[],
  valueAccessor: (dp: ParsedDataPoint) → number | null
) → { min: number | null, max: number | null }
```

Purpose
Scans an array of data points and returns the minimum and maximum valid values.

Rules

- Ignores values that are:
  - `null`
  - `undefined`
  - `NaN`
- Uses `Infinity` / `-Infinity` as initial bounds
- Returns `{ min: null, max: null }` if no valid values are found

Side effects
None (pure function).

`formatValue(v, decimals)`

Signature

```
formatValue(v: unknown, decimals: number | undefined) → string
```

Purpose
Formats values for display.

- `null`, `undefined`, `NaN` → `"N/A"`
- numeric with decimals → fixed decimal string
- otherwise → `String(v)`

Exported Component

`MinMaxReadout({ dataHistory, selectedInterval, nowMs })`

Signature

```
MinMaxReadout(props: {
  dataHistory: ParsedDataPoint[],
  selectedInterval: number,
  nowMs: number
}) → JSX.Element
```

Parameters

- `dataHistory`: full historical array (rolling buffer) from `useShellyStatus`
- `selectedInterval`: time window size in milliseconds (chosen by user)
- `nowMs`: timestamp anchor in milliseconds, typically based on the latest sample time

Returns

- "Awaiting Calculations..." if no data
- Otherwise, a table listing min/max per metric (overall and interval)

Derived Values and Memoization (`useMemo`)

This component uses `useMemo` to avoid expensive recomputation on every render, because min/max scanning can be O(N) per metric.

`grouped`

```
useMemo(() => getGroupedMetricKeys(ALL_METRIC_KEYS), [])
```

Purpose
Computes metric grouping once (catalog is static).

Dependency array explanation

- Empty array because the metric catalog does not change during runtime.

`anchorMs`

```
anchorMs = (nowMs is finite) ? nowMs : Date.now()
```

Used as a stable reference time for filtering.

`cutoff`

```
useMemo(() => anchorMs - selectedInterval, [anchorMs, selectedInterval])
```

Purpose
Computes the earliest allowed timestamp for interval filtering.

Dependency array explanation

- Recompute when the reference time changes (`anchorMs`)
- Recompute when the window length changes (`selectedInterval`)

`filteredData`

```
useMemo(() => dataHistory.filter(...), [dataHistory, cutoff])
```

Purpose
Creates a subset of `dataHistory` containing only points within the selected interval window.

Dependency array explanation

- Recompute when new history arrives (data changes)
- Recompute when cutoff changes (interval or anchor time changes)

`stats`

```
useMemo(() => { ... }, [dataHistory, filteredData])
```

Purpose
Builds an object containing min/max statistics for each metric key:

```
stats[key] = {
  overall: { min, max },
  interval: { min, max }
}
```

Dependency array explanation

- Recompute overall stats when full history changes
- Recompute interval stats when filtered subset changes

Performance note
This is the heaviest computation in this component:

- Loops through each metric key
- For each metric, scans arrays to find min/max
  This is why memoization is important.

Rendering Logic

Loading guard

If `dataHistory` is missing or empty:

- render `Awaiting Calculations...`

Table output

For each group:

- Render metric rows in that group
- Each row shows:
  - parameter label
  - overall min
  - overall max
  - interval min
  - interval max
  - unit

Metric values are extracted via:

- `SERIES_CATALOG[key].valueAccessor(dp)`

Units and formatting come from the registry:

- `m.unit`, `m.decimals`

Side effects

None.

- No timers
- No network
- No state updates beyond rendering

Common Edge Cases

- If a metric produces only null/NaN values:
  - displayed as `N/A`
- If interval window contains no points:
  - interval min/max become `N/A`, but overall still shows values